

Contents

Chapter 4: Evaluation Harness	1
Setup	1
Quick start	1
Commands	2
check	2
run	2
compare	2
gates	3
judge-check	3
new-case	4
--self-check	4
Optional GUI	4
Artifacts and schemas	4
Evaluation vector	4
Project layout	5
Verification	5

Chapter 4: Evaluation Harness

`rag-eval` is an offline evaluation harness for the Chapter 3 RAG pipeline. It validates golden cases, runs the Chapter 3 application through an adapter, computes deterministic retrieval and answer metrics, writes versioned artifacts, compares runs, and evaluates CI gates.

Setup

The lab requires Python 3.12 and `uv`.

```
cd labs/week1/chapter4
uv sync --extra dev
```

The optional GUI dependencies are installed with:

```
uv sync --extra gui
```

No network access, Ollama daemon, or downloaded model is required for the mock workflow.

Quick start

The repository includes a five-case fixture for fast local checks. A production golden dataset should contain substantially more cases; use `--strict` when enforcing the 50-case minimum.

```
# Validate the dataset; writes dataset_report.json by default.
```

```
uv run rag-eval check \
  --dataset tests/fixtures/golden-5.json
```

```
# Run the Chapter 3 pipeline with deterministic mock components.
```

```
uv run rag-eval run \
  --mock \
  --dataset tests/fixtures/golden-5.json \
  --corpus documents \
  --out eval.json
```

```
# Compare two versioned evaluation artifacts.
```

```
uv run rag-eval compare \
```

```
--baseline eval.json \
--current eval.json \
--out compare_report.json
```

The mock run is offline and deterministic. Repeating it with identical inputs produces equivalent metrics and a schema-valid `eval.json` artifact.

Commands

check

Validate a golden dataset without running inference.

```
uv run rag-eval check \
  --dataset path/to/dataset.json \
  [--strict] \
  [--out dataset_report.json]
```

Validation covers required fields, duplicate `case_id` values, closed category membership, `REPLACE_ME` scaffolding sentinels, and—when corpus IDs are supplied by a caller—reference closure. The default mode permits small fixtures. `--strict` rejects datasets with fewer than 50 cases.

Exit codes:

- 0: valid dataset
- 3: dataset violations
- 2: malformed input or usage error

run

Run the Chapter 3 application for every validated case and write an `eval.json` artifact.

```
uv run rag-eval run \
  --dataset path/to/dataset.json \
  --corpus path/to/corpus-or-directory \
  --out eval.json \
  [--mock] \
  [--top-k 5]
```

`--mock` selects the deterministic offline path. The generated artifact contains the report version (0.1), dataset ID, usage kind, per-case verdicts, failure classifications, traces, the aggregate evaluation vector, and `by_category` stratification.

Exit codes:

- 0: evaluation completed
- 3: dataset violations
- 2: missing/unreadable input or usage error

compare

Compare two schema-validated `eval.json` artifacts. The report contains baseline/current values and a direction-aware delta for accuracy, precision, recall, MRR, MAP, NDCG, groundedness, completeness, hallucination rate, latency percentiles, and cost per successful case.

```
uv run rag-eval compare \
  --baseline baseline.json \
  --current current.json \
  [--out compare_report.json] \
  [--force]
```

Higher-is-better metrics use `current - baseline`. Lower-is-better metrics use `baseline - current`. Missing metrics are represented as `n/m`, not zero. Artifacts with mismatched dataset IDs are rejected.

gates

Evaluate hard directional regression constraints from a YAML configuration.

```
uv run rag-eval gates \
  --baseline baseline.json \
  --current current.json \
  --config gates.yml \
  [--out gate_report.json]
```

Example `gates.yml`:

```
version: 1
gates:
- metric: accuracy
  constraint: drop
  max_pct_points: 1.0
- metric: groundedness
  constraint: drop
  max_pct_points: 1.0
- metric: hallucination_rate
  constraint: increase
  max_pct_points: 0.5
- metric: latency_p95
  constraint: increase
  max_pct: 20.0
```

`max_pct_points` is an absolute percentage-point bound for quality metrics. `max_pct` is a relative percentage bound for latency and cost metrics. Unknown metrics, malformed YAML, and missing comparison metrics fail closed.

Exit codes:

- 0: every gate passed
- 1: at least one gate failed
- 2: malformed artifacts or gate configuration

judge-check

Compare verdict fields in an evaluation artifact against human labels. This command is read-only; it does not rewrite `eval.json` or fabricate labels.

```
uv run rag-eval judge-check \
  --eval eval.json \
  --labels labels.json \
  [--out judge_check_report.json]
```

Example `labels.json`:

```
{
  "q-001": {
    "correct": true,
    "supported": true,
    "complete": false,
    "note": "The answer omits the exception."
  }
}
```

```
}  
}
```

The report contains field-level agreement and disagreement pairs. An empty label file returns `NO_LABELS` and exit code 3; ordinary success returns 0.

new-case

Create an incomplete production-to-golden case scaffold from a stored trace.

```
uv run rag-eval new-case \  
  --trace AoEResult.json \  
  --case-id production-001
```

The command prints JSON with `source: production` and `REPLACE_ME` sentinels for `reference_answer`, `relevant_chunks`, and `gold_facts`. A human must complete those fields before the case can pass dataset validation.

--self-check

Verify the deterministic import boundary:

```
uv run rag-eval --self-check
```

The check returns 0 when the deterministic core contains no Chapter 3, Ollama, or network imports. Only `aoe.py` is permitted to cross into the Chapter 3 `rag` package.

Optional GUI

Install the optional dependencies and launch the GUI entry point with:

```
uv sync --extra gui  
uv run rag-eval-gui
```

The GUI entry point is intended for offline artifact browsing and never performs inference. Core CI and test workflows do not require PyQt5.

Artifacts and schemas

The harness uses versioned, schema-gated artifacts:

- `dataset_report.json` — dataset validation outcome and violations
- `eval.json` — per-case traces, verdicts, metrics, and aggregate evaluation vector
- `compare_report.json` — directional metric deltas and category comparisons
- `gate_report.json` — per-gate outcomes and aggregate pass/fail
- `judge_check_report.json` — agreement and disagreement pairs
- `pair_report.json` — reserved format for pairwise evaluation results

Schemas are stored in `schemas/` and include dataset, labels, evaluation, comparison, gates, and pair configuration shapes.

Evaluation vector

The aggregate report includes:

- `accuracy`
- `precision_at_k`, `recall_at_k`, `mrr_at_k`, `map`, `ndcg_at_k`
- `groundedness`, `completeness`, `hallucination_rate`
- `latency_p50`, `latency_p90`, `latency_p95`, `latency_p99`
- `cost_per_success`

- sorted by `by_category` rows and optional `by_difficulty` rows

Zero denominators use documented safe fallbacks. Mock latency and token usage are synthetic and must not be interpreted as production measurements.

Project layout

```
chapter4/
+-- documents/           # local mock corpus
+-- schemas/             # JSON Schemas for all artifact types
+-- src/rag_eval/
|   +-- aoe.py           # sole Chapter 3 adapter boundary
|   +-- dataset.py       # golden dataset types and validation
|   +-- evaluator.py     # deterministic checks and verdict mapping
|   +-- metrics.py       # pure evaluation-vector math
|   +-- failure.py       # failure taxonomy and precedence
|   +-- report.py        # canonical artifact I/O and renderers
|   +-- compare.py       # directional regression comparison
|   +-- gates.py         # fail-closed CI gates
|   +-- judge_check.py   # human-label agreement
|   +-- new_case.py      # production trace scaffolding
|   +-- pair.py          # pairwise result calculation
|   +-- ui.py            # optional read-only GUI entry point
+-- tests/               # offline unit and integration tests
```

Verification

Run the Chapter 4 suite and lint from this directory:

```
unset VIRTUAL_ENV
uv run python -m pytest tests -q
uv run ruff check src tests
uv run rag-eval --self-check
```

The Chapter 4 implementation is designed to wrap, not reimplement, the Chapter 3 RAG pipeline.